Mini Proposal

A. Information

- **Project / Mã và Tên đề tài:** B.10. Using Social Network Structure to Support Content Moderation
- **Course / Môn học:** Mạng xã hội - IS353.R11
- **Duration / Thời gian thực hiện:** 09/2026 - 11/2026
- **Group / Nhóm:** 11
- **Members / Thành viên:** Họ tên, MSSV, email
  1. Lê Gia Quyền, 23521323, 23521323@gm.uit.edu.vn
  2. Đào Bảo Phúc, 23521192, 23521192@gm.uit.edu.vn
  3. Lê Quốc Đại, 23520214, 23520214@gm.uit.edu.vn
  4. Phạm Nguyễn Thanh Sơn, 23521360, 23521360@gm.uit.edu.vn
  5. Trần Tuấn Minh, 23520959, 23520959@gm.uit.edu.vn
  6. Phạm Nguyễn Anh Tuấn, 23521726, 23521726@gm.uit.edu.vn
- **Supervisor / Giảng viên hướng dẫn:** Trần Hưng Nghiệp, nghiepth@uit.edu.vn

B. Content

1. Mini Problem Formulation

Project description. Automated accounts (bots) spread spam and can push a public debate, so platforms flag and review them under their rules on fake and automated accounts. A bot can copy human-like text and profiles, but it may be harder to hide how it is connected to others. This project builds a flag for human review and asks whether an account's interactions add information beyond what it writes and shows in its profile. On the MGTAB dataset [1], we combine content and profile features with features from six types of user-to-user relations, compare content-only, graph-only and combined models under the same split, classifier and imbalance-aware metrics, and use ablations to measure what each signal and relation type adds.

- **Assumption:** the expert labels are correct, and the released features reflect each account's behavior.
- **Constraints:** static snapshot of one debate (Japan's nuclear wastewater release); no raw tweets, only averaged tweet embeddings; 2,748 of 10,199 accounts are bots (26.9%).
- **Scope:** bot vs. human classification of the 10,199 accounts; stance labels are used only to check fairness. **Out of scope:** post-level and raw-text models, harmful-content labels (a bot is not always harmful), real-time use, graph dynamics, explanations.

Existing approaches and research gap. Graph neural network (GNN) bot detectors add relations to profile and text: BotRGCN [2] uses two follow relations, and the MGTAB authors compare GNN types on single relations and some combinations, reporting accuracy only and never combining URL with other relations [1]. Gap: content-only controls, a leave-one-relation-out test and moderation-oriented metrics, and a check of whether errors fall more often on real users from one side of the debate.

- **RQ1:** Does network structure add information beyond content and profile? **H1:** Combined models beat content-only and graph-only baselines, and a relation-aware GNN (P2) beats fixed graph features (P1) and the same GNN on one merged graph, in PR-AUC and macro-F1 (Section 4).
- **RQ2:** Which relation types help most? **H2:** Bots are rarely linked to bots in the interaction relations (follow, mention, reply, quote), but their neighbors through shared URLs and hashtags are bots more often than humans' neighbors are (all labels, Table 7; re-tested on training labels in Exp 4), so these two co-sharing relations bring the largest gain.

Objectives and contributions. A leakage-safe pipeline, baselines, two content + graph models and ablations; a controlled measurement of what each relation type adds; and a moderation-oriented evaluation with a review-budget metric and a fairness check across stance groups.

2. Related Work

Table 1. Related work: short description and drawbacks.

Work	Short description	Drawback / relation to our work
Shi et al. [1]	Source of our dataset; benchmarks many models (bot F1: best GNN, RGT, 90.4 with follower and friend edges; feature-only AdaBoost 87.7); compares single relations and some combinations (accuracy only)	No content-vs-graph control, leave-one-relation-out or fairness check

Work	Short description	Drawback / relation to our work
Feng et al. [2]	BotRGCN: a relational GCN over follower and following edges, with description, tweet and profile features; accuracy 0.846 and F1 0.871 on TwiBot-20 [3]	Two relation types only; our P2 follows its relational GCN design
Song et al. [4]	A relation-specific GNN over the online interaction network, combined with BERT text features, for abusive language detection	Post-level, not accounts
Ribeiro et al. [5]	Hateful-user detection on a Twitter retweet graph (100,386 users); hateful and normal users differ in network structure	One relation type (retweet); our first dataset choice

3. Data

Table 2. Survey of relevant datasets (licenses checked 07–14/10/2026).

Name (source)	Size and stats	License	Short description
MGTAB [1] (GitHub)	10,199 accounts, all labeled (26.9% bots); 1,700,108 edges, 7 types	CC BY-NC-ND 4.0	Twitter; bot and stance labels
Hateful Users on Twitter [5] (Kaggle)	100,386 nodes; 2,286,592 edges; 4,971 labeled (10.9% hateful)	Data CC0	Retweet edges only
TwiBot-20 [3]	229,573 accounts; 11,826 labeled	By request, research only	Follow relations, tweets
Cresci-15 [6]	5,301 accounts (3,351 bots)	Not stated	Follow relations only
BotSim-24 [7]	2,907 accounts (1,000 bots); 46,518 edges	Not stated	Reddit, raw text; simulated bots

We select MGTAB, the only candidate with real accounts, several user-to-user relation types, labels for every account and a size that fits our hardware (statistics in Appendix 3.1). The first dataset, Hateful Users on Twitter, was dropped: one relation type and about ten times more nodes. Like it, MGTAB has no raw tweets.

4. Evaluation

Real life vs. experiment. A deployed system should catch bots (proxy: bot recall), rarely flag real users (precision), rank bots first for a limited review budget (PR-AUC and precision@k, the share of bots among the k highest-scored accounts), work well for both classes (macro-F1), and not flag real users of one stance more often (fairness check: false-positive rate of humans per stance group; stance is never an input).

Metrics. With 26.9% bots, the majority baseline gets 73.1% accuracy but macro-F1 = 0.422, so accuracy is not used. Primary: PR-AUC, the area under the precision–recall curve (bot = positive; random scores give 0.270, about the bot share). Secondary: macro-F1, the mean F1 of the two classes, at the validation-chosen threshold. Also reported: precision@k, bot precision, recall, F1 and ROC-AUC.

Protocol. One stratified 70/15/15 split of the 10,199 accounts (Table 8) for all models; tuning and thresholds on validation only. Model differences are tested with a paired bootstrap [8], and models with random parts run with 3 seeds.

5. Baselines

B1–B2b use Logistic Regression (LR) with the same split, preprocessing and threshold rule, so only the inputs differ: **B0** majority class (sanity check); **B1 content-only**: profile features + LaBSE tweet embeddings [9], with LR and with the MLP of A3 (a nonlinear control for P2); **B2a graph-only**: degree,

PageRank [10], clustering and k-core [11] for each relation, and the follower, friend and list counts and their ratio; **B2b graph-only (stronger)**: Node2Vec embeddings [12]; **B3 label propagation** [13]: spreads training labels over the merged graph. Results of [1] are an external reference only (another split). Details: Appendix 3.3.

6. Proposed Method

We add network context to content in two steps (Figure 1, Appendix 3.4). **P1 – Fusion-LR** joins content, profile and graph features (structural + Node2Vec) in the same LR as the baselines, so any gain comes from combining the sources. **P2 – RGCN (main model)** [14] combines each account's features with its neighbors' features, using a separate weight matrix for each relation type, so follow links and shared-URL links can count differently. P2 follows the relational GCN design of [2]; our contribution is the controlled test of what each signal and relation adds, not a new architecture.

Why it should work. Bots link to others differently: they have 14.8 follow neighbors on average against 83.5 for humans, and their URL and hashtag neighbors are bots more often than humans' neighbors (0.291 vs. 0.186 and 0.352 vs. 0.236; Table 7). One merged graph mixes these patterns; separate weights keep them apart. In [1], BotRGCN reaches higher bot accuracy with several relations than with any single one (87.2 vs. at most 83.5).

7. Expected Results

We expect (1) every feature-based baseline to beat B0 (B3 may not, since bots are rarely linked to bots); (2) P1 to beat B1, B2a, B2b and B3, and P2 to beat P1, its multi-layer perceptron (MLP) version A3 and its merged-graph version A4 (H1); (3) removing the URL or hashtag relation to cost P2 the most (H2). The fairness gap is measured, not predicted; the bot share differs by stance (neutral 33.7%, against 24.4%, support 21.1%). Experiments: Appendix 3.6; no prediction of gain sizes.

Backup plan. If P2 fails, the ablations show where the gain is lost; since bots rarely link to bots, message passing may add little over A3, which still answers RQ2. A fairness gap counts only if its bootstrap interval excludes zero; per-group thresholds are then tested as analysis only, since stance is not known at decision time. Differences not supported by the paired bootstrap are not counted. The LR models run on a CPU, so RQ1 is answered even if the RGCN fails.

References

[1] S. Shi *et al.*, "MGTAB: A multi-relational graph-based Twitter account detection benchmark," 2023, arXiv:2301.01123v2, doi: 10.48550/arXiv.2301.01123.

[2] S. Feng, H. Wan, N. Wang, and M. Luo, "BotRGCN: Twitter bot detection with relational graph convolutional networks," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining (ASONAM)*, 2021, pp. 236–239, doi: 10.1145/3487351.3488336.

[3] S. Feng, H. Wan, N. Wang, J. Li, and M. Luo, "TwiBot-20: A comprehensive Twitter bot detection benchmark," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2021, pp. 4485–4494, doi: 10.1145/3459637.3482019.

[4] R. Song, F. Giunchiglia, Q. Shen, N. Li, and H. Xu, "Improving abusive language detection with online interaction network," *Inf. Process. Manage.*, vol. 59, no. 5, Art. no. 103009, 2022, doi: 10.1016/j.ipm.2022.103009.

[5] M. H. Ribeiro, P. H. Calais, Y. A. Santos, V. A. F. Almeida, and W. Meira Jr., “Characterizing and detecting hateful users on Twitter,” in *Proc. Int. AAAI Conf. Web Social Media (ICWSM)*, vol. 12, no. 1, 2018, pp. 676–679, doi: 10.1609/icwsm.v12i1.15057.

[6] S. Cresci, R. Di Pietro, M. Petrocchi, A. Spognardi, and M. Tesconi, “Fame for sale: Efficient detection of fake Twitter followers,” *Decis. Support Syst.*, vol. 80, pp. 56–71, 2015, doi: 10.1016/j.dss.2015.09.003.

[7] B. Qiao, K. Li, W. Zhou, S. Li, Q. Lu, and S. Hu, “BotSim: LLM-powered malicious social botnet simulation,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 13, 2025, pp. 14377–14385, doi: 10.1609/aaai.v39i13.33575.

[8] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994, doi: 10.1201/9780429246593.

[9] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, “Language-agnostic BERT sentence embedding,” in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, 2022, pp. 878–891, doi: 10.18653/v1/2022.acl-long.62.

[10] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the Web,” Stanford InfoLab, Tech. Rep. 1999-66, 1999. [Online]. Available: http://ilpubs.stanford.edu:8090/422/

[11] S. B. Seidman, “Network structure and minimum degree,” *Social Netw.*, vol. 5, no. 3, pp. 269–287, 1983, doi: 10.1016/0378-8733(83)90028-X.

[12] A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864, doi: 10.1145/2939672.2939754.

[13] X. Zhu and Z. Ghahramani, “Learning from labeled and unlabeled data with label propagation,” Carnegie Mellon Univ., Tech. Rep. CMU-CALD-02-107, 2002. [Online]. Available: https://www.cs.cmu.edu/~zhuxj/pub/CMU-CALD-02-107.pdf

[14] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling relational data with graph convolutional networks,” in *The Semantic Web: ESWC 2018* (Lecture Notes in Computer Science, vol. 10843). Springer, 2018, pp. 593–607, doi: 10.1007/978-3-319-93417-4_38.

[15] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” presented at the ICLR Workshop Represent. Learn. Graphs Manifolds, 2019, arXiv:1903.02428. [Online]. Available: https://arxiv.org/abs/1903.02428

Appendix

1. Timeline

The project has five stages aligned with the course milestones: (1) proposal and registration, (2) data preparation and network analysis, (3) modeling and evaluation, (4) consolidation for Progress Review 2, and (5) final quality control and pre-defense.

Table 3. Project timeline aligned with course milestones (Weeks 2–11).

Stage	Week	Detailed tasks	Responsible
1	Week 2	Finalize topic, research problem and scope; project registration; set up shared repository, task board and report template	All; Quyền leads; Tuấn (setup)
1	Week 3	Write proposal sections: problem formulation (Quyền), related work and baselines (Đại), dataset survey and license check (Phúc), evaluation and expected results (Minh), proposed method, ablation and experiment specifications (Sơn); integrate, control length, QC and references (Tuấn); proposal review; change of dataset to MGTAB after supervisor feedback	Quyền (integration); all
2	Week 4	Load the MGTAB graph; remove self-loops and duplicate edges; merge follower and friend into one follow relation; define feature groups; fix the stratified 70/15/15 split; audit data leakage	Phúc (data), Minh (split protocol), Tuấn (leakage audit)
2	Week 5	Network analysis per relation (degree distribution, PageRank, clustering, k-core); Node2Vec embeddings; implement baselines B0–B3	Phúc (network analysis); Đại (Node2Vec, baselines)
2	Week 6	Progress Review 1: present dataset, network analysis and preliminary findings; revise based on feedback	All; Quyền coordinates; Tuấn (consistency check)
3	Week 7	Finalize baselines; implement evaluation pipeline (PR-AUC, macro-F1, precision@k, paired bootstrap, fairness check) and class-imbalance handling; leakage checks including the zero-tweet accounts	Đại, Minh; Tuấn (leakage checks)
3	Week 8	Implement P1, P2 (RGCN) and the MLP (A3); first experiments	Sơn; Đại supports
3	Week 9	Compare baselines and proposed models; ablations A1–A8; repeat runs with 3 seeds (Exp 1–3); relation analysis (Exp 4); error analysis, fairness and moderation checks (Exp 5–6); result tables and figures	Minh (evaluation, Exp 6), Sơn (models, Exp 5); Phúc (Exp 4, tables, figures)
4	Week 10	Progress Review 2: consolidate results, figures, tables and discussion; verify all reported numbers against logs	All; Quyền coordinates; Tuấn (verification)
5	Week 11	Final QC, Word formatting and length control, references, slides, pre-defense preparation	Quyền (format, length), Tuấn (QC, references); all

Table 4. Member responsibilities and task ownership.

Member	Main responsibility
Lê Gia Quyền	Lead writer: problem logic, writing style, length and Word format control; Part A, Section 1 and review of the whole document
Đào Bảo Phúc	Dataset search, license, size and structure; Section 3; Appendix 3.1; network analysis; result tables and figures
Lê Quốc Đại	Literature search; baselines B0–B3; Sections 2 and 5; Appendix 3.3
Phạm Nguyễn Thanh Sơn	Content + network architecture and its rationale; Section 6; Appendix 3.4–3.6

Member	Main responsibility
Trần Tuấn Minh	Metrics for imbalanced data; real-life KPIs vs. experimental metrics; evaluation protocol and fairness check; Sections 4 and 7; Appendix 3.2
Phạm Nguyễn Anh Tuấn	QC, timeline and project management, data-leakage checks, verification of AI-generated content; Appendix 1 and references

Feasibility. Six members work on separate workstreams (data, related work and baselines, method, evaluation, QC) in parallel, so modeling in Weeks 7–9 starts from a validated data pipeline and a fixed evaluation protocol. The graph is small (10,199 accounts, about 0.9 million user pairs), so every model runs on our own laptops. Progress Reviews in Weeks 6 and 10 are checkpoints to find technical risks and reduce scope if needed, for example by dropping some ablations.

2. Additional content

Table 5 shows real rows of the released data. Profile values are scaled to [0, 1] by the dataset authors; the tweet embedding has 768 values (only the first is shown). Neighbor counts are computed by us.

Table 5. Data sample from features.pt, labels_bot.pt, labels_stance.pt and the edge files.

Index	Bot label	Stance	Verified	Followers (scaled)	Statuses (scaled)	Tweet emb. [0]	Follow neighbors	URL neighbors
0	human	against	0	0.2081	0.1697	1.134	56	0
1	human	support	0	0.4027	0.6984	0.584	7	0
3	bot	neutral	0	0.0620	0.3555	0.814	0	195
7	bot	neutral	0	0.1402	0.4394	1.022	0	245

3. Experimental plan (proposal version 2)

3.1. Dataset

Table 6. Dataset statistics (MGTAB [1]).

Property	Value
Accounts (nodes)	10,199, all labeled; collected around the debate on Japan's nuclear wastewater release; each labeled by nine annotators from a team of 12 experts, majority vote [1]
Bot label	2,748 bots, 7,451 humans (26.9% bots)
Stance label (fairness check only)	neutral 3,776; against 3,637; support 2,786
Features per account	788: 20 profile features + 768-dimensional mean LaBSE embedding of the account's tweets [9] (54 languages, 73.6% English [1])
Edge rows (raw)	1,700,108 in 7 types, incl. 205,819 self-loops and 251,448 duplicate rows
Follower vs. friend	95.7% of follower edges are friend edges in the opposite direction, so we merge them into one follow relation (6 relations)
All relations merged (undirected)	902,856 user pairs; 54 accounts without any edge
Possible shortcuts (tested in A8)	387 accounts with an all-zero tweet embedding and 239 verified accounts; all of them are human

Table 7. The six relations after cleaning (all labels, for description only).

Relation	Edges (directed)	User pairs	Accounts with ≥1 edge	Mean neighbors: human / bot	Bot share among neighbors: of bots / of humans
Follow	425,863	331,251	9,443	83.5 / 14.8	0.010 / 0.065
Mention	110,868	107,794	7,538	26.8 / 5.7	0.004 / 0.078
Reply	22,327	20,700	3,604	5.5 / 0.3	0.008 / 0.019
Quote	20,901	20,536	3,652	5.2 / 0.7	0.002 / 0.051
URL (shared)	undirected	263,800	6,251	56.1 / 39.9	0.291 / 0.186

Relation	Edges (directed)	User pairs	Accounts with ≥1 edge	Mean neighbors: human / bot	Bot share among neighbors: of bots / of humans
Hashtag (shared)	undirected	300,000	9,588	59.0 / 58.3	0.352 / 0.236

Table 8. Train / validation / test statistics (stratified split, random_state = 42).

Split	Accounts	Bots	Humans	Bot %
Train (70%)	7,139	1,924	5,215	27.0%
Validation (15%)	1,530	412	1,118	26.9%
Test (15%)	1,530	412	1,118	26.9%
Total	10,199	2,748	7,451	26.9%

Table 9. Feature groups: the 788 released features plus 156 features we compute.

Group	Columns	Count	Use
P: profile	Yes/no fields (background image, default profile, verified, default profile image, geo enabled, four default-color/URL fields, has URL); statuses and favourites counts; account creation time; screen name, name and description length	16	B1, P1, P2
T: content	Mean LaBSE embedding of the account's tweets [9]	768	B1, P1, P2
G-struct	Profile counts: followers, friends, listed, followers/friends ratio (given). Per relation (ours): in- and out-degree, PageRank, clustering, k-core for follow, mention, reply, quote; degree, PageRank, clustering, k-core for URL and hashtag	4 + 28	B2a, P1, P2
G-emb	Node2Vec embedding of the merged graph (ours)	128	B2b, P1
Not features	Bot label (target); stance label (fairness check only); edge weights (not used)	–	Never model inputs

Data pipeline.

1. Load the six .pt files with PyTorch: features, bot and stance labels, edge list, edge types and weights; row i of every file is account i.
2. Split the accounts into train / validation / test first (Table 8) and fix the split for all experiments.
3. Remove self-loops and duplicate edges; merge follower and friend into one follow relation (Table 7).
4. Compute the structural features of each relation, and 128-dimensional Node2Vec embeddings on the merged undirected graph: 10 walks of length 80 per account and window size 10, as in [12]; p and q in {0.5, 1, 2}, chosen on validation PR-AUC of B2b. No labels are used.
5. Group the columns as in Table 9. The profile features are already scaled to [0, 1] by the authors (fitted on all accounts, without labels); the LaBSE embedding is not scaled. Our computed features are standardized with statistics from training accounts only.

3.2. Evaluation Protocol

Evaluation data. All 10,199 accounts are labeled; every model is evaluated on the same test set (Table 8).

Evaluation metrics. The bot class is positive. With TP, FP and FN the true positives, false positives and false negatives:

$$Precision = \frac{TP}{TP + FP}, Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}, Macro - F1 = \frac{1}{2}\left(F1_{bot} + F1_{human}\right)$$

PR-AUC is computed as Average Precision (sklearn.metrics.average_precision_score), where P_n and R_n are the precision and recall at the n-th threshold:

$$AP = \sum_n \left(R_n - R_{n-1}\right) P_n$$

Reference values: random scores give PR-AUC = 0.270 and ROC-AUC ≈ 0.50 (mean of 100 runs), and B0 gives macro-F1 = 0.422. **Precision@k** is the share of bots among the k highest-scored test accounts (k = 100 and 200; the test set has 412 bots). **Fairness check:** for each stance group, the false-positive rate FP / (FP + TN) of test humans, and the largest gap between groups with a 95% bootstrap interval (each group has only a few hundred test humans).

Evaluation pipeline.

1. Use the fixed split and the preprocessed features from 3.1.
2. Train each model; tune hyperparameters on validation PR-AUC (the RGCN also uses it for early stopping).
3. Choose the decision threshold that maximizes validation macro-F1.
4. Evaluate once on the test set: PR-AUC, macro-F1, precision@k, bot-class precision, recall and F1, ROC-AUC, and the fairness check.
5. Report 95% bootstrap confidence intervals (1,000 resamples of the test accounts) for every model. To compare two models, use a paired bootstrap on the difference (the same resampled accounts for both) [8]. Models with random parts (Node2Vec, RGCN, MLP) also report mean ± standard deviation over 3 seeds.

Leakage control. Labels of validation and test accounts are never used for training, tuning, preprocessing or features. Graph features and models may use the whole graph structure, because they use no labels (see the transductive setting in 3.4). Stance labels are never model inputs. Label-based network statistics in Exp 4 use training labels only.

3.3. Baseline

B1–B2b use LR with the same split, preprocessing, threshold rule and metrics, so their differences come only from the input features. B3 is a feature-free graph baseline.

Table 10. Baseline configurations.

Baseline	Input features (Table 9)	Model
B0: Majority	None	Always predicts "human"
B1: Content-only	P + T (profile, LaBSE)	LR; and MLP (same grid as A3)
B2a: Graph-only	G-struct (structure of each relation and profile counts)	LR
B2b: Graph-only (stronger)	G-emb (Node2Vec, 128-d)	LR
B3: Label propagation	Training labels and the merged undirected graph; no features	Label propagation [13]

Training. LR uses class weights from the training class ratio, the same imbalance handling as P2. The regularization strength $C \in \{0.01, 0.1, 1, 10\}$ is tuned on validation PR-AUC (a smaller grid than the 8

settings of the neural models, which we note when comparing them). B3 keeps the training labels fixed and repeats neighbor averaging until the scores change by less than 10^{-6} (at most 100 rounds); the score is the propagated bot share. B0 needs no training.

3.4. Proposed Method

Main idea. Keep the classifier fixed and add graph features (P1), then let the model learn from the neighbors' features, separately for each relation (P2). Comparing P1 with the baselines isolates the value of graph information; comparing P2 with its MLP version (A3) and its merged-graph version (A4) isolates the value of message passing and of relation types.

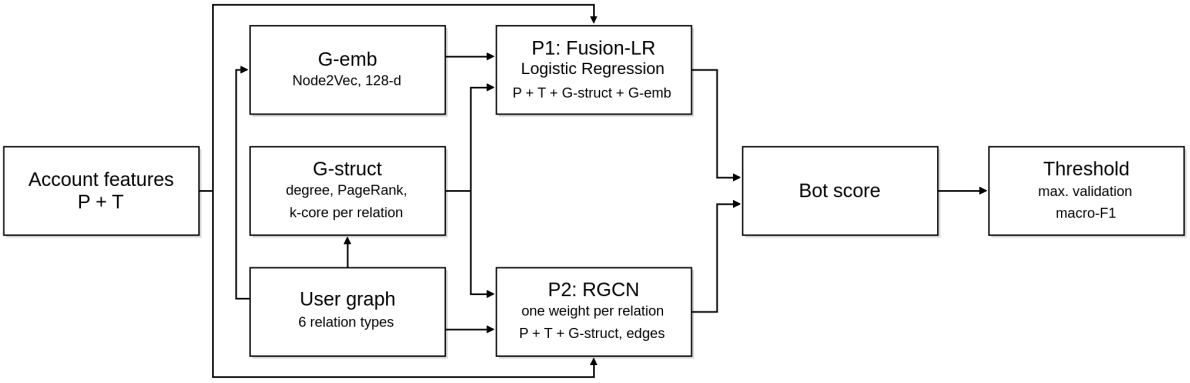


Figure 1. Proposed models. P1 uses P + T + G-struct + G-emb; P2 uses P + T + G-struct and the typed edges.

Graph. The 10,199 accounts with the six cleaned relations (Table 7). Inside each relation, edges are treated as undirected, so an account aggregates, for example, both the users it mentions and the users who mention it.

P1 – Fusion-LR. Input P + T + G-struct + G-emb; LR with the same settings and tuning as B1 and B2.

P2 – RGCN. Two layers [14]. For account v, R the set of six relations and $N_r(v)$ the neighbors of v in relation r, layer k computes:

$$h_v^k = ReLU\left(W_0^k h_v^{k-1} + \sum_{r \in R} \sum_{u \in N_r(v)} \frac{1}{\left|N_r(v)\right|} W_r^k h_u^{k-1} \right), h_v^0 = x_v$$

A linear layer on $h_v^{(2)}$ gives the bot score. The input x_v is P + T + G-struct. G-emb is not used, because the RGCN learns the neighborhood structure itself.

Training. Transductive: the model sees the features and edges of all 10,199 accounts, but the loss uses only training labels. This matches real moderation, where the platform knows the whole graph but not who is a bot; Node2Vec in B2b also uses the whole graph. Class-weighted cross-entropy (weights from the training class ratio), Adam, early stopping on validation PR-AUC. Initial settings, tuned on validation: hidden size 128, dropout 0.5, learning rate 0.01, up to 200 epochs. The decision threshold of P1 and P2 is chosen on validation to maximize macro-F1. The graph (10,199 nodes × 816 inputs, about 0.9M user pairs) fits full-batch training on a 4 GB GPU or a CPU, with PyTorch Geometric [15].

3.5. Ablation

All ablations use the same split, seeds, threshold rule and metrics as the main models.

Table 11. Ablation settings.

No.	Setting	What changes	Question answered
A1	P2 − T	Remove the tweet embedding	Is content needed when neighbors are used?
A2	P1 − G-emb; P1 − G-struct	Remove Node2Vec, or remove the structural features	Which graph features bring the gain?
A3	MLP on P2 inputs	Same inputs, 2 hidden layers, same grid and early stopping as P2; no neighbor aggregation	Does message passing itself help?
A4	P2 on one merged relation; P2 on the 7 released relations	All edges share one weight matrix; or follower and friend kept apart	Do relation types matter, and does the merge lose information?
A5	P2 minus one relation	Six runs, each without one relation	Which relation matters most? (H2)
A6	P2 with interaction or co-sharing relations only	Follow, mention, reply, quote vs. URL, hashtag	Which kind of link carries the signal?
A7	P2 without class weights	Plain cross-entropy	Does imbalance handling help?
A8	P1, P2 without zero-tweet or verified accounts	Remove the 387 zero-tweet and the 239 verified accounts from training and test	Do results depend on these shortcuts?

3.6. Experiment specifications

Common settings: the split of Table 8; seeds 0, 1 and 2 (they affect Node2Vec, the RGCN and the MLP; LR is deterministic); hyperparameters, early stopping and thresholds chosen on validation only; one setting changed at a time; 95% bootstrap intervals on the test set. Each run saves its configuration, library versions (Python, scikit-learn, PyTorch, PyTorch Geometric) and test metrics to a results file, so every reported number can be traced to a run.

Table 12. Experiment specifications and expected outcomes.

Experiment	Question	Models / settings	Output	Expected outcome
Exp 1: Baselines	How strong is each signal alone? (RQ1)	B0, B1 (LR and MLP), B2a, B2b, B3; C tuned on validation	Metrics table with 95% bootstrap intervals	Feature-based baselines clearly exceed B0 in PR-AUC and macro-F1; B3 may not
Exp 2: Proposed models	Does graph information add to content? (RQ1, H1)	P1, P2; RGCN and MLP grid: hidden {64, 128}, dropout {0.3, 0.5}, learning rate {0.005, 0.01}	Metrics table; paired bootstrap vs. Exp 1	P1 beats B1, B2a, B2b and B3; P2 beats P1 (H1)
Exp 3: Ablations	Which part brings the gain? (RQ1, RQ2)	A1–A8 (Table 11), with the best Exp 2 settings	Metric change vs. the full model	A1, A3 and A4 lower performance; removing URL or hashtag costs most (A5, H2); class weights raise bot recall (A7); results hold without the shortcut accounts (A8)
Exp 4: Relation analysis	How do bots connect? (RQ2, H2)	Training labels only: degree of bots vs. humans per relation (Mann-Whitney U test); bot share among the training neighbors of training bots and humans vs. the base rate; humans with at least one bot neighbor (exposure). Analysis only, never a model input	Statistics table and distribution plots	Lower interaction degree for bots; bot neighbors above the base rate only in URL and hashtag (H2)

Experiment	Question	Models / settings	Output	Expected outcome
Exp 5: Error and fairness analysis	Where do the models fail, and for whom?	Test errors of P1 and P2 grouped by degree, relation coverage and zero-tweet accounts; false-positive rate of humans per stance group, with bootstrap intervals	Error and fairness tables	No prediction; a gap leads to the per-group threshold analysis (Section 7)
Exp 6: Moderation check	Would the model help moderators?	Precision@100 and @200 on the test set for all models	KPI table	Top-scored accounts are bots far more often than the 26.9% base rate